

Sophie Li

sophielidev@gmail.com |(617)594-2496 | San Jose, CA

SUMMARY

- Senior Software Engineer with **10+ years** of experience in **Spring Boot 3.x** and **Java 17**, specializing in high-concurrency distributed backend systems across e-commerce, fintech, and content platforms.
- Expert in designing and managing **Spring Cloud Gateway** and **Kong** clusters with custom Global Filters for identity propagation, dynamic request transformation, and rate limiting, processing 1M+ daily requests across **RESTful** microservices.
- Specialized in distributed patterns including Saga orchestration, Transactional Outbox, and Redis-based idempotency for high-consistency order and payment workflows.
- Proficient in financial systems engineering with **Spring Batch**, BigDecimal precision, and automated reconciliation pipelines, ensuring PCI-DSS / SOC 2 compliance.
- Skilled in high-throughput data modeling on **Cassandra** LSM-tree storage with compound partition keys, **MySQL** with covering indexes and read/write splitting, and multi-tier caching with Caffeine and **Redis**.
- Hands-on with cloud-native deployment on **AWS** (EKS, RDS, S3, MSK, Lambda, SQS) using **Docker**, **Kubernetes**, **Terraform**, and CI/CD pipelines with Jenkins and GitHub Actions.
- Advanced implementation of resilience and observability with **Resilience4j** circuit breaker, bulkhead, and rate limiting, backed by Prometheus, Grafana, and ELK monitoring stack.
- Architected secure backend services with **Spring Security**, OAuth 2.0, JWT, and RBAC, enabling single sign-on (SSO) and delegated authorization for internal admin tools.

SKILLS

Programming Languages: Java 8/11/17/21, JavaScript, TypeScript, SQL, Python, Bash

Backend Frameworks: Spring Boot, Spring Cloud, Spring Security, Spring AOP, Spring WebFlux, Spring Batch, Spring Data JPA, Spring MVC, Netty

APIs & Messaging: RESTful APIs, GraphQL, gRPC, WebSocket, Kafka, Flink, RabbitMQ, Zookeeper, OpenAPI/Swagger

Databases & Storage: MySQL, PostgreSQL, MongoDB, Redis, Cassandra, Elasticsearch

Cloud & DevOps: AWS (EC2, RDS, S3, EKS, ECR, ECS, VPC, ELB, SNS, SQS, CloudWatch, IAM, API Gateway), Azure, GCP, Docker, Kubernetes, Terraform, Jenkins, GitHub Actions, Maven, Git

Testing & Performance: JUnit 5, Mockito, Spring Boot Test, MockMvc, Postman, JMeter, RestAssured, TestNG, Cucumber, Selenium, SonarQube

Observability & Logging: Prometheus, Grafana, ELK Stack, Kibana, Micrometer, Logback, CloudWatch, Datadog

Security: Spring Security, OAuth2, JWT, RBAC

Frontend: React, Angular, TypeScript, HTML5, CSS3, Axios, Node.js

EXPERIENCE

Wayfair

03/2023 – Present

Senior Software Engineer

Project Description:

Contributed as a senior engineer to the order service of a large-scale e-commerce platform handling order placement, payment processing, inventory deduction, and order lifecycle management. Built with Java 17 and Spring Boot, integrated with upstream and downstream services via OpenFeign and Kafka. The system processes high-volume transactions daily with strict consistency and high availability requirements, supporting peak traffic scenarios during large-scale promotional events.

Responsibilities:

- Designed and owned **REST APIs** for order creation, status transitions, and query flows, defining service boundaries across cart, inventory, and fulfillment services to reduce cross-service coupling for 100K+ daily transactions.
- Implemented **Saga** orchestration via **OpenFeign** to coordinate inventory, product, and payment workflows, designing compensating transactions and retry mechanisms to maintain **eventual consistency** during downstream failures.
- Designed order status lifecycle using a state machine with **idempotency** controls, eliminating duplicate transitions under high-concurrency retries and reducing order inconsistency incidents by 30%.
- Built a **Kafka**-based event-driven pipeline for order lifecycle events, processing 500K+ order events daily with average end-to-end latency under **200ms**.
- Implemented the **Transactional Outbox** pattern to persist order state changes and domain events in the same database transaction, improving Kafka publishing reliability during service restarts and failure scenarios.
- Optimized **MySQL** queries and **indexing** strategies, reducing P99 latency from 800ms to under 200ms under peak load.
- Implemented **Redis caching** for frequently accessed data, reducing database load and cutting MySQL read **QPS** by **40%** during peak traffic.

- Designed a **Cassandra** historical order archive partitioned by `userId` with clustering on `orderTime DESC`; offloaded 200M+ orders from MySQL, reducing primary database size by 60% with order-history queries under 50ms p99.
- Implemented **JWT** authentication and **RBAC** at the API Gateway, integrating **AWS IAM** for access control and identity management across microservices.
- Applied **Resilience4j circuit breakers**, **retries**, and **timeouts** on critical service dependencies, preventing cascading failures and improving overall service availability during peak traffic.
- Designed multi-layer **rate limiting** at the API Gateway using Spring Cloud Gateway and Redis, protecting services from burst traffic and maintaining system stability under high-concurrency workloads.
- Parallelized downstream service calls using `CompletableFuture` with **thread pool** isolation, reducing end-to-end API response latency by 40%.
- Implemented async **audit logging** via **Spring AOP** for full order state-transition traceability, reducing mean time to diagnose production incidents by 50%.
- Architected a **MongoDB** audit and order-history store with flexible document schema, a compound index on `orderId` and `timestamp`, and TTL-based retention; built **aggregation pipelines** for analytics on 50M+ historical events.
- Built monitoring dashboards using **Prometheus**, **Grafana** and **AWS CloudWatch**, and centralized logging with **ELK** to support production **observability** and **alerting**.
- Owned **CI/CD** pipelines using **Jenkins** and **GitHub Actions** to automate build, test, and deployment workflows.
- Established automated testing standards with **JUnit 5**, **Mockito**, **Spring Boot Test**, and **MockMvc**, enforcing PR quality gates and maintaining **80%** code coverage.
- Containerized microservices with **Docker** and deployed on **AWS EKS**, configuring auto-scaling and health checks to maintain 99.9% service availability under variable load.
- Built reactive aggregation APIs with **Spring WebFlux** and **Reactor Mono/Flux**, fanning out non-blocking parallel calls to inventory, pricing, and recommendation services; reduced p99 latency by **45%** at **5K+ RPS**.
- Build a **GraphQL** BFF layer with **DataLoader** for batch resolution and N+1 query prevention, aggregating data from order, inventory, and product services to serve mobile and web clients with a unified schema.

Wealthfront

12/2019 – 12/2022

Senior Software Engineer

Project Description:

Contributed as a senior engineer to a large-scale Wealth Management Platform, responsible for fixed-income subscription, order lifecycle management, and daily earnings calculation. The system supports end-to-end financial workflows from user purchase to settlement and earnings display, designed with strict consistency, high availability, and financial-grade data accuracy requirements.

Responsibilities:

- Owned the fixed-income subscription service, designing **REST APIs** for purchase initiation, share allocation, and order lifecycle management; enforced idempotency and optimistic locking to prevent duplicate transactions under high concurrency.
- Enforced **transaction** boundaries across subscription and earnings workflows with state transition validation and **compensation** handling, ensuring data consistency under partial-failure scenarios.
- Built **Spring Batch** pipelines for daily earnings calculation across 1M+ user accounts, ensuring accurate and consistent balance updates with zero financial discrepancy.
- Built position and earnings inquiry service with **Redis caching** and cache invalidation coordinated with Spring Batch job completion, reducing MySQL read QPS by **35%** during peak access windows.
- Designed **MySQL sharding** strategy partitioning data by user ID with read/write separation, supporting high-concurrency workloads and improving query throughput by 30%.
- Implemented financial accuracy safeguards using **BigDecimal** for all monetary calculations; built **reconciliation** jobs to detect order state mismatches and automated alerts to reduce financial discrepancy incidents.
- Built automated ingestion of daily custodian via **AWS S3** and **Lambda**, triggering Spring Batch earnings calculations and **SNS/SES** alerts to operations teams on processing completion and anomalies.
- Built **Kafka**-based event pipelines for subscription workflows with **idempotent** consumers and **DLQ** handling, ensuring reliable event processing under at-least-once delivery for 400K+ daily subscription events.
- Optimized query performance through **indexing** and **query routing** strategies, reducing key query latency by **30%** under peak load.
- Set up **Prometheus** and **Grafana** monitoring for the subscription service, tracking key **metrics** such as batch job execution, reconciliation discrepancies, and subscription success rate; implemented alerting to detect data issues early and support faster incident response.
- Owned **CI/CD** pipelines via **Jenkins** and **GitHub Actions** with branch-based deployment gates across dev, staging, and prod, reducing release cycle time and improving deployment reliability.
- Implemented **pre-release validation**, **rollback mechanisms**, and dependency checks to improve release stability and reduce production risks.
- Partnered with customer service and operations teams on production issue triage; leveraged distributed tracing and trace IDs to track cross-service call chains, identify root causes of subscription failures, and reduce mean time to resolution.

Project Description:

Contributed as a backend engineer to short-video platform, responsible for the video upload service, metadata management, content status tracking, and creator-facing content APIs. The platform supported high-throughput video uploads, multi-format encoding, and content distribution across web and mobile.

Responsibilities:

- Owned the video upload and metadata service, implementing resumable upload APIs and creator-facing metadata APIs with **idempotency** controls, input validation, and **versioned endpoints**.
- Designed and implemented the video lifecycle **state machine** with strict transition guards and idempotency controls; prevented duplicate state transitions under concurrent worker updates and resolved a class of race conditions that caused videos to get stuck in intermediate processing states.
- Built **Kafka**-based video event pipeline with manual offset acknowledgment, idempotency checks, and DLQ handling to ensure reliable event delivery between encoding workers and downstream consumers.
- Optimized **MySQL** query performance for the metadata service: profiled slow queries, added composite indexes, and rewrote inefficient query patterns; reduced read query latency by **32%** under peak traffic.
- Built multi-level **Redis** caching for hot video metadata and feed result sets with tuned **TTL** and cache invalidation on status transitions — cut tail latency on read-heavy feed endpoints by **30%** during peak traffic.
- Designed a **Cassandra**-backed engagement aggregation service for likes, views, and comments, optimizing partition key design for even data distribution and horizontal scalability without impacting the MySQL write path.
- Partnered with search and recommendation teams to expose watch-time and interaction signals via Kafka event streams, enabling downstream content ranking features and improving content engagement metrics.
- Set up **Elasticsearch** log aggregation and **Kibana** dashboards for real-time anomaly detection; reduced mean time to diagnose production failures **from 2 hours to under 40 minutes** via structured log search and cross-service trace correlation.
- Applied **Resilience4j circuit breakers** with **fallback** strategies on external service dependencies to prevent cascading failures during upstream degradation and improve overall service availability.
- Led **on-call** rotation for the video upload and metadata service, resolving incidents including transcoding failures, videos stuck in intermediate states, and upload callback timeouts; documented root causes to prevent recurrence.
- Established **JUnit 5** and **Mockito** testing standards for the content backend team: authored guidelines covering service-layer mock testing and Kafka consumer test patterns; lifted team code coverage **from 45% to 75%**.
- Improved metadata consistency across **asynchronous** content workflows by strengthening state transition validation and worker callback handling, reducing stale intermediate states after processing failures.
- Enforced backward-compatibility checks for metadata APIs and event schemas across content services, reducing downstream breakages during rolling deployments and cross-service upgrades.
- Improved production readiness for the content ingestion service: standardized pre-release validation, dependency checks, and rollback procedures for major traffic events, reducing deployment risk during peak content ingestion periods.

EDUCATION

Northeastern University

Master of Science – Software Engineering

University of Electronic Science and Technology of China

Bachelor of Science – Computer Science